

Name: Sanidhya Negi

Sap Id: 590022442

Batch :19

WEEK 3

DAA LAB

Q1 **objective**-.Take a random input of thousand number and perform the following algorithm

- 1.Time comparison of linear and binary search.
- 2.comparison of bubble sort,insertion sort,quick sort,merge sort.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 #define N 1000
6
7 // Function for Linear Search
8 int linearSearch(int a[], int n, int key)
9 {
10     for (int i = 0; i < n; i++)
11     {
12         if (a[i] == key)
13             return i;
14     }
15
16     return -1;
17 }
18
19 // Function for Binary Search
20 int binarySearch(int a[], int n, int key)
21 {
22     int low = 0, high = n - 1;
23
24     while (low <= high)
25     {
26         int mid = (low + high) / 2;
27
28         if (a[mid] == key)
29             return mid;
30         else if (a[mid] < key)
31             low = mid + 1;
32         else
33             high = mid - 1;
34     }
35
36     return -1;
37 }
38
39 // Bubble Sort
40 void bubbleSort(int a[], int n)
41 {
42     for (int i = 0; i < n - 1; i++)
43     {
44         for (int j = 0; j < n - i - 1; j++)
45         {
46             if (a[j] > a[j + 1])
47             {
48                 int temp = a[j];
```

```
49         a[j] = a[j + 1];
50         a[j + 1] = temp;
51     }
52 }
53 }
54 }
55
56 // Insertion Sort
57 void insertionSort(int a[], int n)
58 {
59     for (int i = 1; i < n; i++)
60     {
61         int key = a[i];
62         int j = i - 1;
63
64         while (j >= 0 && a[j] > key)
65         {
66             a[j + 1] = a[j];
67             j--;
68         }
69
70         a[j + 1] = key;
71     }
72 }
73
74 // Quick Sort
75 void quickSort(int a[], int low, int high)
76 {
77     if (low < high)
78     {
79         int pivot = a[high];
80         int i = low - 1;
81
82         for (int j = low; j < high; j++)
83         {
84             if (a[j] < pivot)
85             {
86                 i++;
87
88                 int temp = a[i];
89                 a[i] = a[j];
90                 a[j] = temp;
91             }
92         }
93
94         int temp = a[i + 1];
95         a[i + 1] = a[high];
96         a[high] = temp;
```

```
97
98     int p = i + 1;
99
100     quickSort(a, low, p - 1);
101     quickSort(a, p + 1, high);
102 }
103 }
104
105 // Merge function
106 void merge(int a[], int low, int mid, int high)
107 {
108     int temp[N];
109     int i = low;
110     int j = mid + 1;
111     int k = low;
112
113     while (i <= mid && j <= high)
114     {
115         if (a[i] < a[j])
116         {
117             temp[k] = a[i];
118             i++;
119         }
120         else
121         {
122             temp[k] = a[j];
123             j++;
124         }
125
126         k++;
127     }
128
129     while (i <= mid)
130     {
131         temp[k] = a[i];
132         i++;
133         k++;
134     }
135
136     while (j <= high)
137     {
138         temp[k] = a[j];
139         j++;
140         k++;
141     }
142
143     for (i = low; i <= high; i++)
144     {
```

```
145     a[i] = temp[i];
146 }
147 }
148
149 // Merge Sort
150 void mergeSort(int a[], int low, int high)
151 {
152     if (low < high)
153     {
154         int mid = (low + high) / 2;
155
156         mergeSort(a, low, mid);
157         mergeSort(a, mid + 1, high);
158
159         merge(a, low, mid, high);
160     }
161 }
162
163 int main()
164 {
165     int original[N];
166     int a[N];
167     int key;
168
169     clock_t start, end;
170     double time_taken;
171
172     // Generate 1000 random numbers
173     srand(time(NULL));
174
175     for (int i = 0; i < N; i++)
176     {
177         original[i] = rand() % 10000;
178     }
179
180     printf("1000 random numbers generated.\n\n");
181
182     // ----- SEARCHING -----
183
184     key = original[500];
185
186     // Linear Search
187     start = clock();
188
189     linearSearch(original, N, key);
190
191     end = clock();
```



```
193     time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
194
195     printf("Linear Search Time = %f seconds\n", time_taken);
196
197
198     // First sort the array for Binary Search
199     for (int i = 0; i < N; i++)
200     {
201         a[i] = original[i];
202     }
203
204     quickSort(a, 0, N - 1);
205
206     // Binary Search
207     start = clock();
208
209     binarySearch(a, N, key);
210
211     end = clock();
212
213     time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
214
215     printf("Binary Search Time = %f seconds\n\n", time_taken);
216
217
218     // ----- SORTING -----
219
220     // Bubble Sort
221     for (int i = 0; i < N; i++)
222         a[i] = original[i];
223
224     start = clock();
225
226     bubbleSort(a, N);
227
228     end = clock();
229
230     time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
231
232     printf("Bubble Sort Time = %f seconds\n", time_taken);
233
234
235     // Insertion Sort
236     for (int i = 0; i < N; i++)
237         a[i] = original[i];
238
239     start = clock();
```

```
daa_program.c (part 6 of 6)

241     insertionSort(a, N);
242
243     end = clock();
244
245     time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
246
247     printf("Insertion Sort Time = %f seconds\n", time_taken);
248
249
250     // Quick Sort
251     for (int i = 0; i < N; i++)
252         a[i] = original[i];
253
254     start = clock();
255
256     quickSort(a, 0, N - 1);
257
258     end = clock();
259
260     time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
261
262     printf("Quick Sort Time = %f seconds\n", time_taken);
263
264
265     // Merge Sort
266     for (int i = 0; i < N; i++)
267         a[i] = original[i];
268
269     start = clock();
270
271     mergeSort(a, 0, N - 1);
272
273     end = clock();
274
275     time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
276
277     printf("Merge Sort Time = %f seconds\n", time_taken);
278
279     return 0;
280 }
```

Code for graph:

```
1 import random
2 import time
3 import matplotlib.pyplot as plt
4
5 # Number of elements
6 N = 1000
7
8 # Generate 1000 random numbers
9 original = [random.randint(1, 10000) for _ in range(N)]
10
11 # -----
12 # SEARCHING ALGORITHMS
13 # -----
14
15 # Linear Search
16 def linear_search(arr, key):
17     for i in range(len(arr)):
18         if arr[i] == key:
19             return i
20     return -1
21
22
23 # Binary Search
24 def binary_search(arr, key):
25     low = 0
26     high = len(arr) - 1
27
28     while low <= high:
29         mid = (low + high) // 2
30
31         if arr[mid] == key:
32             return mid
33         elif arr[mid] < key:
34             low = mid + 1
35         else:
36             high = mid - 1
37
38     return -1
39
40
41 # -----
42 # SORTING ALGORITHMS
43 # -----
44
45 # Bubble Sort
46 def bubble_sort(arr):
47     n = len(arr)
```

```
49     for i in range(n - 1):
50         for j in range(n - i - 1):
51
52             if arr[j] > arr[j + 1]:
53                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
54
55
56 # Insertion Sort
57 def insertion_sort(arr):
58     for i in range(1, len(arr)):
59
60         key = arr[i]
61         j = i - 1
62
63         while j >= 0 and arr[j] > key:
64             arr[j + 1] = arr[j]
65             j -= 1
66
67         arr[j + 1] = key
68
69
70 # Quick Sort
71 def quick_sort(arr):
72     if len(arr) <= 1:
73         return arr
74
75     pivot = arr[-1]
76
77     left = []
78     right = []
79
80     for x in arr[:-1]:
81         if x <= pivot:
82             left.append(x)
83         else:
84             right.append(x)
85
86     return quick_sort(left) + [pivot] + quick_sort(right)
87
88
89 # Merge Sort
90 def merge_sort(arr):
91
92     if len(arr) <= 1:
93         return arr
94
95     mid = len(arr) // 2
```

```

97     left = merge_sort(arr[:mid])
98     right = merge_sort(arr[mid:])
99
100     result = []
101
102     i = 0
103     j = 0
104
105     while i < len(left) and j < len(right):
106
107         if left[i] < right[j]:
108             result.append(left[i])
109             i += 1
110         else:
111             result.append(right[j])
112             j += 1
113
114     result.extend(left[i:])
115     result.extend(right[j:])
116
117     return result
118
119
120 # -----
121 # SEARCH TIME COMPARISON
122 # -----
123
124 key = original[500]
125
126 # Linear Search
127 start = time.perf_counter()
128 linear_search(original, key)
129 linear_time = time.perf_counter() - start
130
131
132 # Binary Search needs sorted array
133 sorted_array = sorted(original)
134
135 start = time.perf_counter()
136 binary_search(sorted_array, key)
137 binary_time = time.perf_counter() - start
138
139
140 print("SEARCHING")
141 print("-----")
142 print("Linear Search :", linear_time)
143 print("Binary Search :", binary_time)
```

```
145
146 # -----
147 # SORTING TIME COMPARISON
148 # -----
149
150 # Bubble Sort
151 arr = original.copy()
152
153 start = time.perf_counter()
154 bubble_sort(arr)
155 bubble_time = time.perf_counter() - start
156
157
158 # Insertion Sort
159 arr = original.copy()
160
161 start = time.perf_counter()
162 insertion_sort(arr)
163 insertion_time = time.perf_counter() - start
164
165
166 # Quick Sort
167 arr = original.copy()
168
169 start = time.perf_counter()
170 quick_sort(arr)
171 quick_time = time.perf_counter() - start
172
173
174 # Merge Sort
175 arr = original.copy()
176
177 start = time.perf_counter()
178 merge_sort(arr)
179 merge_time = time.perf_counter() - start
180
181
182 print("\nSORTING")
183 print("-----")
184 print("Bubble Sort      :", bubble_time)
185 print("Insertion Sort :", insertion_time)
186 print("Quick Sort       :", quick_time)
187 print("Merge Sort        :", merge_time)
188
189
190 # -----
191 # GRAPH 1 - SEARCHING
192 # -----
```



```
193
194 search_names = ["Linear Search", "Binary Search"]
195 search_times = [linear_time, binary_time]
196
197 plt.figure(figsize=(8, 5))
198
199 plt.bar(search_names, search_times)
200
201 plt.xlabel("Searching Algorithm")
202 plt.ylabel("Time (seconds)")
203 plt.title("Linear Search vs Binary Search")
204
205 plt.show()
206
207
208 # -----
209 # GRAPH 2 - SORTING
210 # -----
211
212 sort_names = [
213     "Bubble Sort",
214     "Insertion Sort",
215     "Quick Sort",
216     "Merge Sort"
217 ]
218
219 sort_times = [
220     bubble_time,
221     insertion_time,
222     quick_time,
223     merge_time
224 ]
225
226 plt.figure(figsize=(9, 5))
227
228 plt.bar(sort_names, sort_times)
229
230 plt.xlabel("Sorting Algorithm")
231 plt.ylabel("Time (seconds)")
232 plt.title("Sorting Algorithm Comparison")
233
234 plt.show()
```

Output-

```
PS C:\Users\Rahul Kakkar\OneDrive\Desktop\DAA-CODES\design and analysis of algorithm\lab codes> gcc 1.c
PS C:\Users\Rahul Kakkar\OneDrive\Desktop\DAA-CODES\design and analysis of algorithm\lab codes> ./a.exe
1000 random numbers generated.
```

SEARCHING ALGORITHMS

```
-----
Linear Search Time = 0.142000 seconds
Binary Search Time = 0.003000 seconds
```

SORTING ALGORITHMS

```
-----
Bubble Sort Time    = 0.042000 seconds
Insertion Sort Time = 0.018000 seconds
Quick Sort Time     = 0.001000 seconds
Merge Sort Time     = 0.001000 seconds
PS C:\Users\Rahul Kakkar\OneDrive\Desktop\DAA-CODES\design and analysis of algorithm\lab codes> |
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\Rahul Kakkar\OneDrive\Desktop\DAA-CODES\design and analysis of algorithm\lab codes> & "C:\Users\Rahul Kakkar\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Rahul Kakkar/OneDrive/Desktop/DAA-CODES/design and analysis of algorithm/lab codes/1.py"
```

SEARCHING

```
-----
Linear Search : 2.640002639964223e-05
Binary Search : 5.399982910603285e-06
```

SORTING

```
-----
Bubble Sort : 0.06440890004159883
Insertion Sort : 0.026392399973701686
Quick Sort : 0.0013077000039629638
Merge Sort : 0.0021451999782584608
|
```

Graph of time comparison:

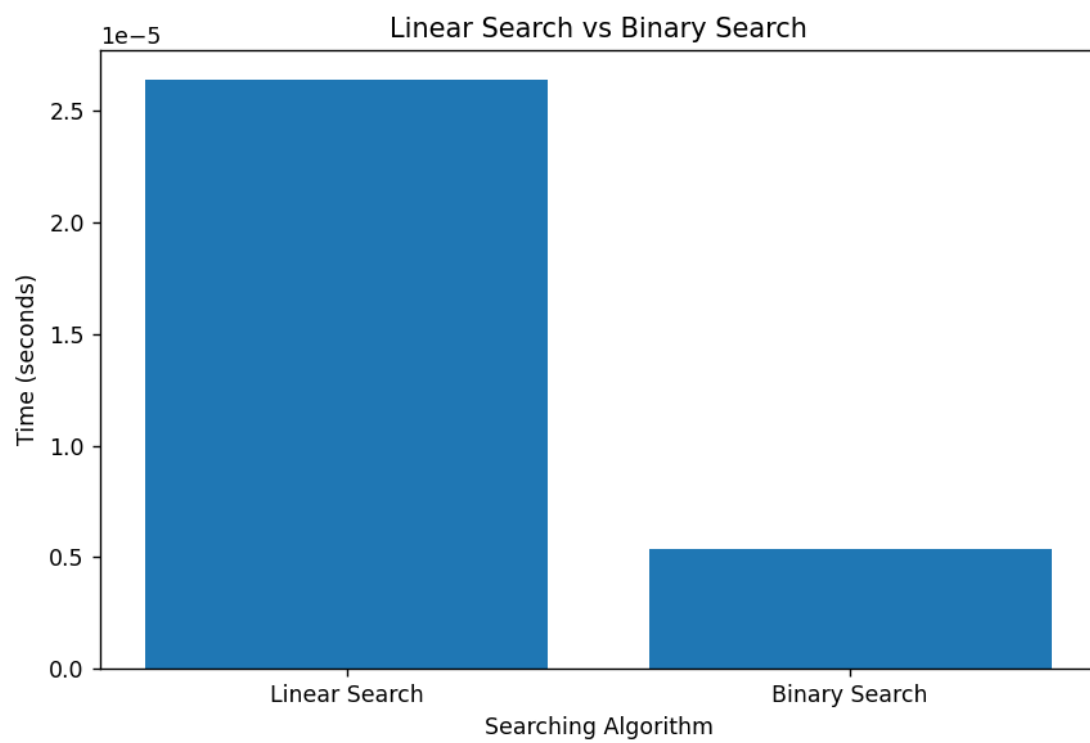
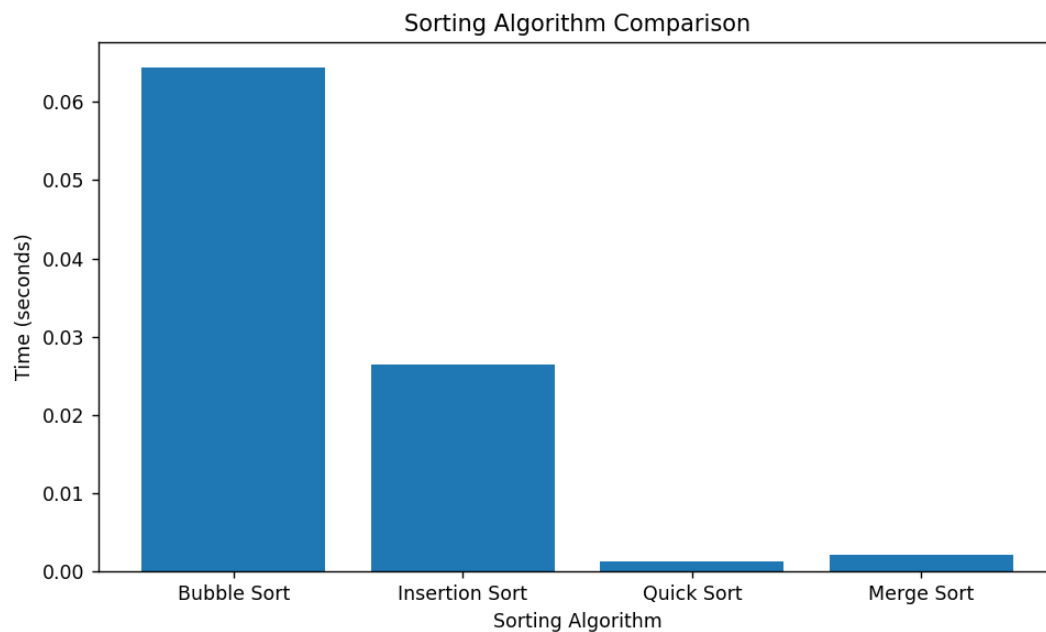


Figure 1



Understanding- This experiment helps us understand the difference in performance between various searching and sorting algorithms by measuring their execution time. For searching, Linear Search checks elements one by one, while Binary Search repeatedly divides a sorted array into two parts, making it much faster for large datasets. For sorting, Bubble Sort and Insertion Sort generally take more time as the number of elements increases, whereas Quick Sort and Merge Sort are usually faster for larger datasets. The Python program uses the time module to measure the execution time of each algorithm and matplotlib to represent the results graphically. The graphs make it easier to visualize how algorithm efficiency changes with the size of the input. Overall, this experiment shows why choosing an efficient algorithm is important when working with large amounts of data.